Informe Taller Práctico – Contenedores (Docker)

Pontificia Universidad Javeriana · Ingeniería de Sistemas · Arquitectura de Software

Se desplegaron por consola tres contenedores conectados en la red `red-fis`: PostgreSQL 18.6, MySQL 8.4.13 y CloudBeaver Community 26.2.3. Cada motor guarda sus datos en un volumen propio, y CloudBeaver se conectó a ambos por nombre de contenedor.

Elemento	Detalle
Sistema	Windows con Docker Desktop (backend WSL 2), consola CMD
Docker	Engine 29.1.5 · Compose v5.0.1
Red	`red-fis` (bridge, 172.25.0.0/16)
Volúmenes	`pg_fis_data` · `mysql_fis_data` · `cb_fis_workspace`

Captura 1 · Instalación y verificación de Docker

```
C:\Users\juand>docker --version
Docker version 29.1.5, build 0e6fee6

C:\Users\juand>docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

El mensaje describe el ciclo completo cliente → demonio → registro → contenedor. Esta captura es de una segunda ejecución de `hello-world`, y responde la pregunta de clase: ya no aparece *Unable to find image… Pulling*, porque la imagen quedó guardada localmente en la primera ejecución (visible con `docker images`). Docker solo creó un contenedor nuevo a partir de ella y lo eliminó al terminar (`--rm`).

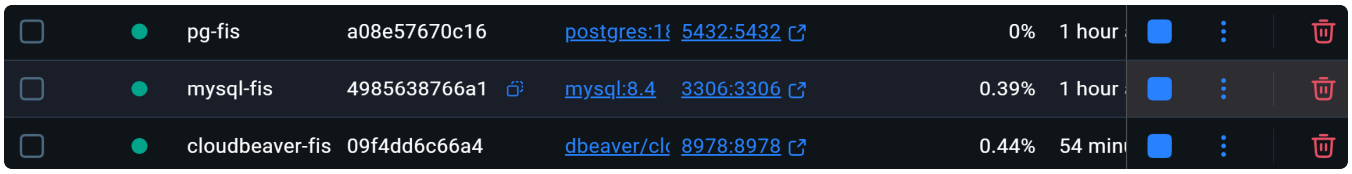
Captura 2 · Contenedores y red

```
C:\Users\juand>docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED             STATUS             PORTS                                                NAMES
09f4dd6c66a4   dbeaver/cloudbeaver:latest     "./launch-product.sh"    48 minutes ago      Up 48 minutes      0.0.0.0:8978->8978/tcp, [::]:8978->8978/tcp          cloudbeaver-fis
4985638766a1   mysql:8.4                      "docker-entrypoint.s…"   57 minutes ago      Up 57 minutes      0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp          mysql-fis
a08e57670c16   postgres:18                    "docker-entrypoint.s…"   About an hour ago   Up About an hour   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp          pg-fis

C:\Users\juand>docker network inspect red-fis
[
    {
        "Name": "red-fis",
        "Id": "44f07d0f9c0cde8a3d7924618ba8d67d8caaccca03a6b15e1f68e33d4df6504f",
        "Created": "2026-10-29T20:47:25.308525702Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.25.0.0/16",
                    "IPRange": "",
                    "Gateway": "172.25.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {
            "09f4dd6c66a4abba071e9c0ed02f690e0d32995fcfdc44d0c4b3606885e4f182": {
                "Name": "cloudbeaver-fis",
                "EndpointID": "5c32ce4b33da28e9d5aa3ee4ac0f1567ef128dad7aa1b13db74627bd2be59d01",
                "MacAddress": "2e:a9:7c:a3:56:22",
                "IPv4Address": "172.25.0.4/16",
                "IPv6Address": ""
            },
            "4985638766a112ac6bba5e5f4770cb7238e7cdb246dd1fbb0ead96c45b53e962": {
                "Name": "mysql-fis",
                "EndpointID": "5c56f1c9bda9521ff1b1a06390fc3eeb679a0b5bea71a4a254eafca9a798ac42",
                "MacAddress": "9a:4a:9d:61:25:e7",
                "IPv4Address": "172.25.0.3/16",
                "IPv6Address": ""
            },
            "a08e57670c169dc9aa2e0c043a78320c04b8ebd7f6c8a86ca12be76db3865615": {
                "Name": "pg-fis",
                "EndpointID": "bf8e3c57a119c7df832ccd050ea6bee19e68acc117a274b58655b98ea0a873a8",
                "MacAddress": "ce:72:15:48:13:e6",
                "IPv4Address": "172.25.0.2/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.25.0.0/16": {
                        "IPsInUse": 6,
                        "DynamicIPsAvailable": 65530
                    }
                }
            }
        }
    }
]

C:\Users\juand>
```

La misma información en la interfaz de Docker Desktop: los tres contenedores en ejecución con sus imágenes y puertos publicados.



Los tres contenedores están en estado **Up** y comparten la red `red-fis`, donde cada uno recibe su propia IP:

Contenedor	Imagen	IP en red-fis	Puerto publicado
`pg-fis`	`postgres:18`	172.25.0.2	5432 → 5432
`mysql-fis`	`mysql:8.4`	172.25.0.3	3306 → 3306
`cloudbeaver-fis`	`dbeaver/cloudbeaver:latest`	172.25.0.4	8978 → 8978

Captura 3 · Superusuario y privilegios

PostgreSQL: `admin_fis` tiene los atributos Superuser, Create role, Create DB, Replication y Bypass RLS.

```
C:\Users\juand>docker exec -it pg-fis psql -U admin_fis -d fis_db
psql (18.6 (Debian 18.6-1.pgdg13+2))
Type "help" for help.

fis_db=# \du
                            List of roles
 Role name |                         Attributes
-----------+------------------------------------------------------------
 admin_fis | Superuser, Create role, Create DB, Replication, Bypass RLS

fis_db=# SELECT rolname, rolsuper, rolcreatedb FROM pg_roles WHERE rolname = 'a
dmin_fis';
  rolname  | rolsuper | rolcreatedb
-----------+----------+-------------
 admin_fis | t        | t
(1 row)
```

MySQL – usuarios creados: `root` y `app_fis` con host `%` (pueden conectarse desde otros contenedores, como CloudBeaver).

```
C:\Users\juand>docker exec -it mysql-fis mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.4.13 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT user, host FROM mysql.user;
+------------------+-----------+
| user             | host      |
+------------------+-----------+
| app_fis          | %         |
| root             | %         |
| mysql.infoschema | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
+------------------+-----------+
6 rows in set (0.00 sec)

mysql>
```

MySQL – root: todos los privilegios sobre `*.*` con GRANT OPTION.

```
mysql> SHOW GRANTS FOR 'root'@'%'\G
*************************** 1. row ***************************
Grants for root@%: GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, RELOAD,
SHUTDOWN, PROCESS, FILE, REFERENCES, INDEX, ALTER, SHOW DATABASES, SUPER, CREAT
E TEMPORARY TABLES, LOCK TABLES, EXECUTE, REPLICATION SLAVE, REPLICATION CLIENT
, CREATE VIEW, SHOW VIEW, CREATE ROUTINE, ALTER ROUTINE, CREATE USER, EVENT, TR
IGGER, CREATE TABLESPACE, CREATE ROLE, DROP ROLE ON *.* TO `root`@`%` WITH GRAN
T OPTION
*************************** 2. row ***************************
Grants for root@%: GRANT ALLOW_NONEXISTENT_DEFINER,APPLICATION_PASSWORD_ADMIN,A
UDIT_ABORT_EXEMPT,AUDIT_ADMIN,AUTHENTICATION_POLICY_ADMIN,BACKUP_ADMIN,BINLOG_A
DMIN,BINLOG_ENCRYPTION_ADMIN,CLONE_ADMIN,CONNECTION_ADMIN,ENCRYPTION_KEY_ADMIN,
FIREWALL_EXEMPT,FLUSH_OPTIMIZER_COSTS,FLUSH_PRIVILEGES,FLUSH_STATUS,FLUSH_TABLE
S,FLUSH_USER_RESOURCES,GROUP_REPLICATION_ADMIN,GROUP_REPLICATION_STREAM,INNODB_
REDO_LOG_ARCHIVE,INNODB_REDO_LOG_ENABLE,OPTIMIZE_LOCAL_TABLE,PASSWORDLESS_USER_
ADMIN,PERSIST_RO_VARIABLES_ADMIN,REPLICATION_APPLIER,REPLICATION_SLAVE_ADMIN,RE
SOURCE_GROUP_ADMIN,RESOURCE_GROUP_USER,ROLE_ADMIN,SENSITIVE_VARIABLES_OBSERVER,
SERVICE_CONNECTION_ADMIN,SESSION_VARIABLES_ADMIN,SET_ANY_DEFINER,SHOW_ROUTINE,S
YSTEM_USER,SYSTEM_VARIABLES_ADMIN,TABLE_ENCRYPTION_ADMIN,TELEMETRY_LOG_ADMIN,TR
ANSACTION_GTID_TAG,XA_RECOVER_ADMIN ON *.* TO `root`@`%` WITH GRANT OPTION
2 rows in set (0.00 sec)
```

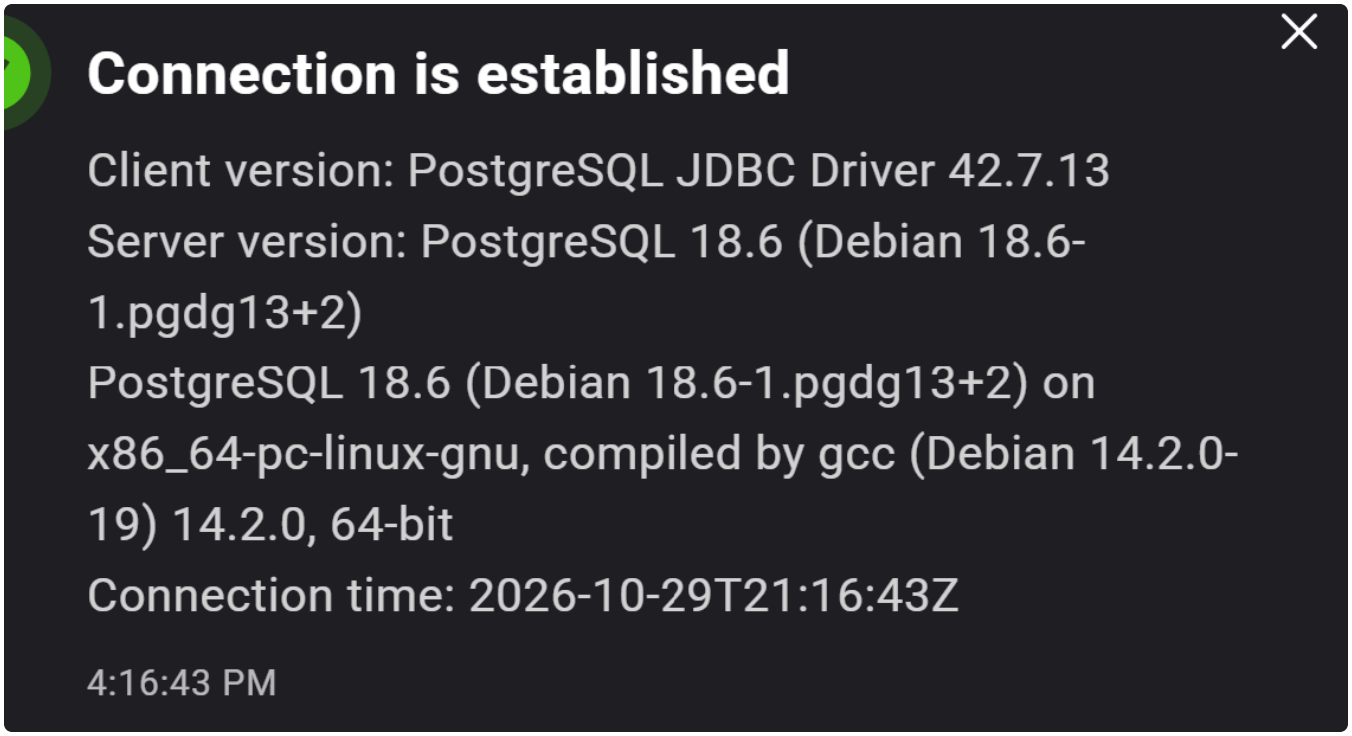
MySQL – app_fis: solo `USAGE` global y `ALL PRIVILEGES` sobre `fis_db.*`.

```
mysql> SHOW GRANTS FOR 'app_fis'@'%'\G
*************************** 1. row ***************************
Grants for app_fis@%: GRANT USAGE ON *.* TO `app_fis`@`%`
*************************** 2. row ***************************
Grants for app_fis@%: GRANT ALL PRIVILEGES ON `fis\_db`.* TO `app_fis`@`%`
2 rows in set (0.00 sec)
```

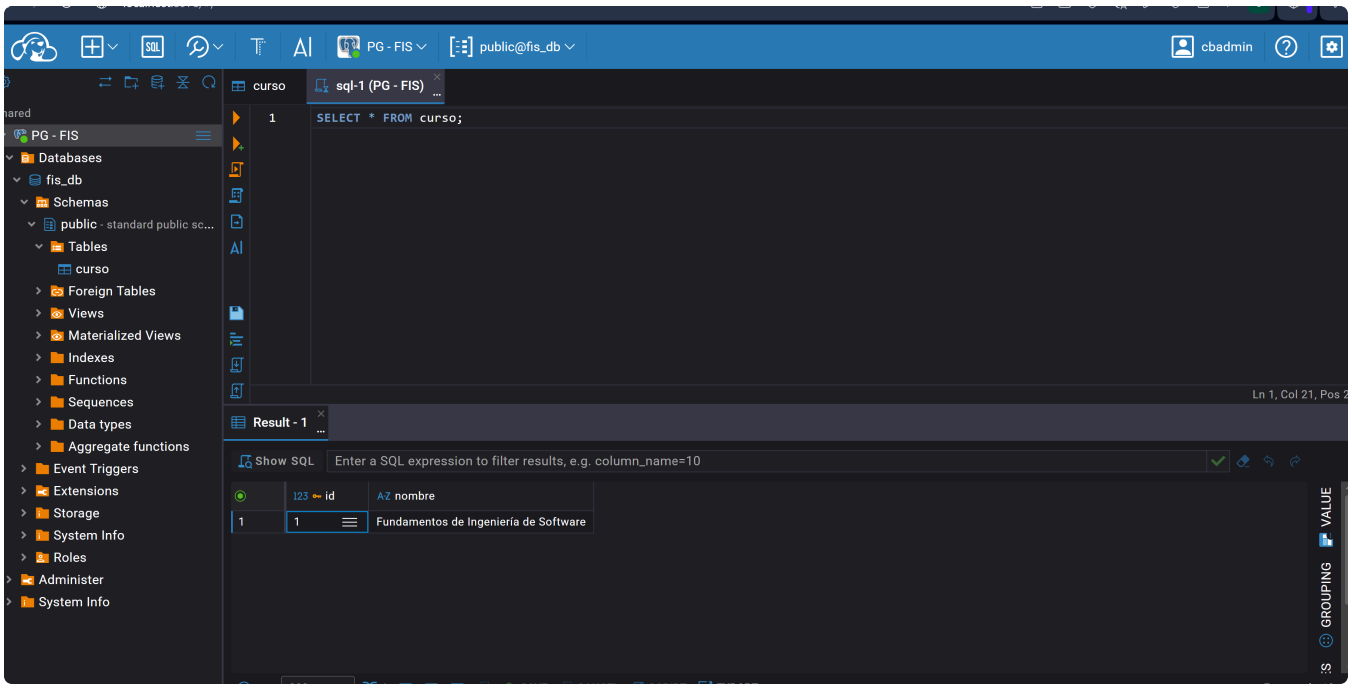
Explicación de privilegios. En PostgreSQL, `admin_fis` es superusuario y controla todo el servidor. En MySQL, `root` tiene todos los privilegios sobre todas las bases (`*.*`) con GRANT OPTION, por lo que puede crear usuarios y delegar permisos. En cambio, `app_fis` solo puede conectarse a nivel global (`USAGE`) y tiene privilegios completos únicamente sobre `fis_db`. Por eso las aplicaciones deben usar usuarios como `app_fis`: si su contraseña se filtra, el daño queda limitado a una sola base.

Captura 4 · CloudBeaver conectado a ambos motores

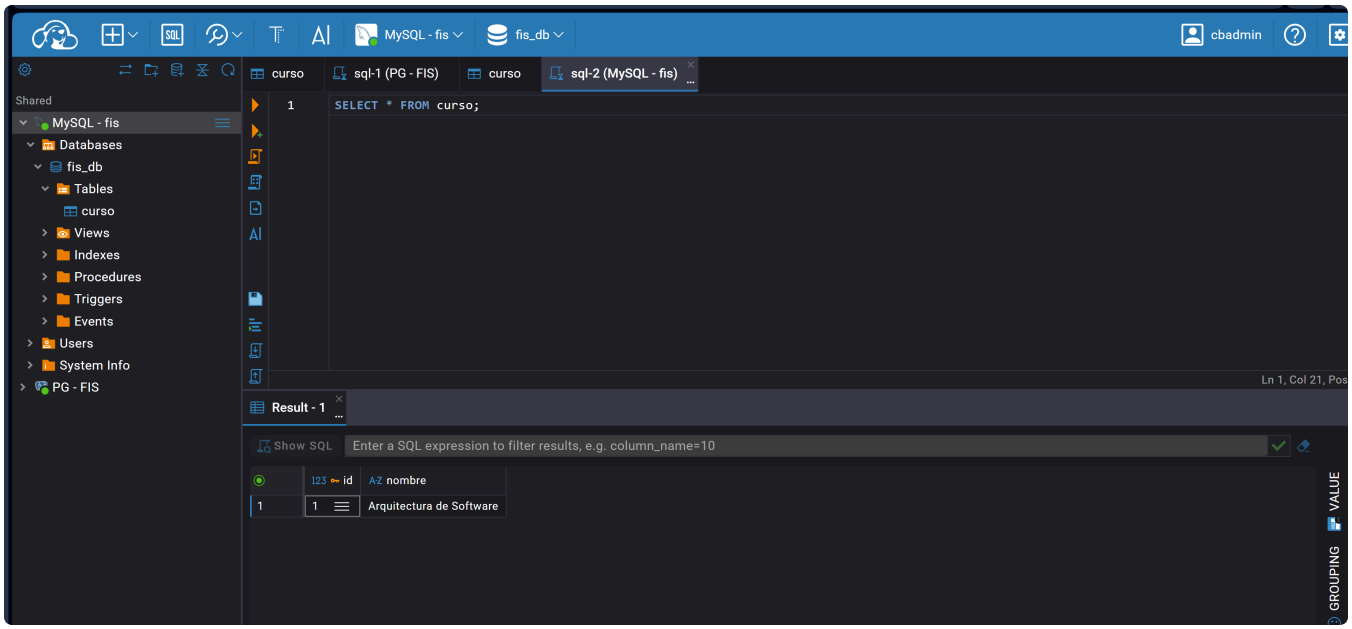
Prueba de conexión exitosa a PostgreSQL usando el host `pg-fis` dentro de `red-fis`:



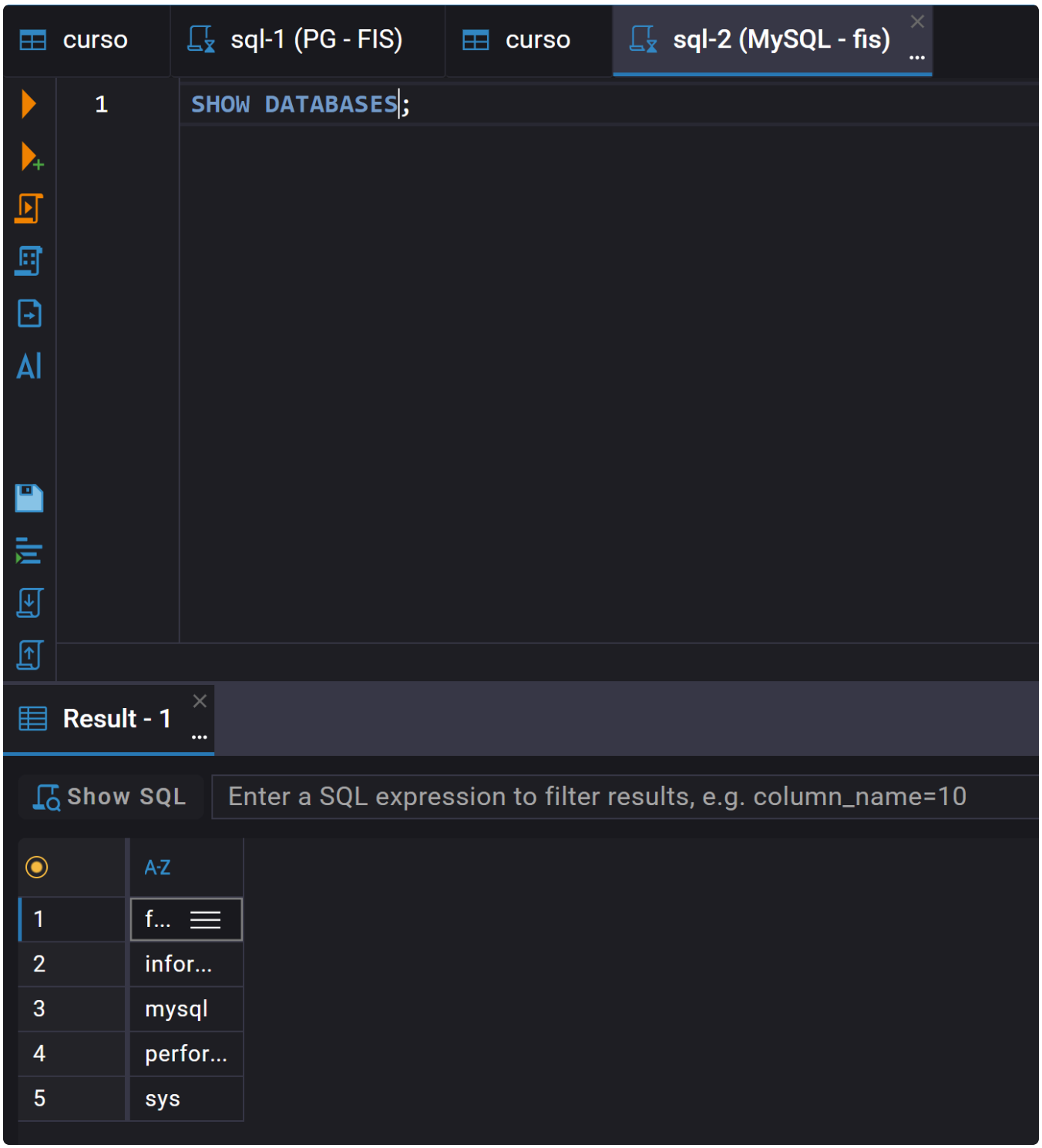
`SELECT * FROM curso;` en **PG - FIS**:



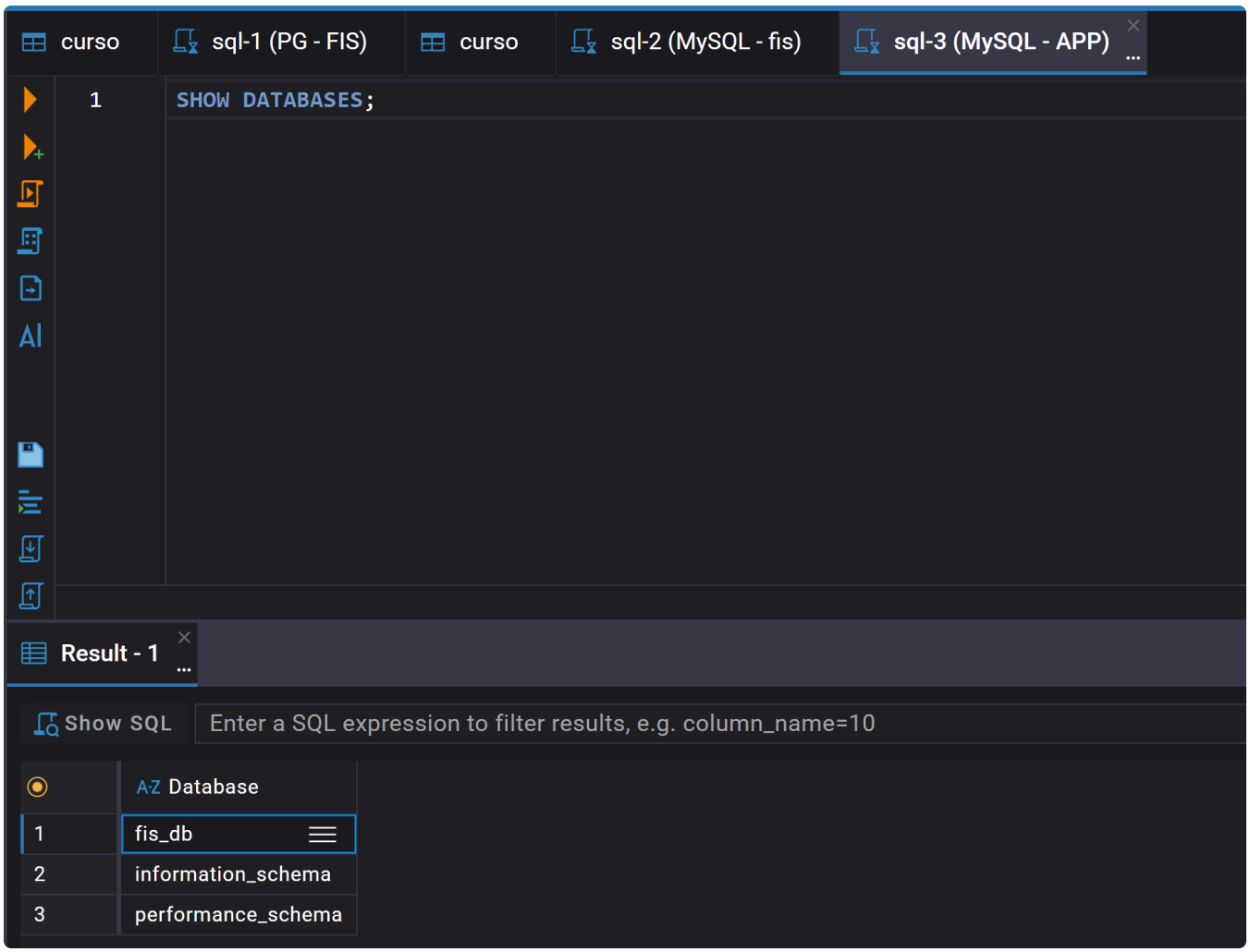
`SELECT * FROM curso;` en **MySQL - fis** (root):



Contraste de alcance (`SHOW DATABASES;`). Con `root` se ven 5 bases, incluidas `mysql` y `sys`:



Con `app_fis` (conexión **MySQL - APP**) solo se ven 3, y no aparecen `mysql` ni `sys`, donde están las cuentas y la configuración del servidor:



Punto 5 · Reto: ¿por qué pg-fis y no localhost?

CloudBeaver se conecta a `pg-fis:5432` y no a `localhost:5432` porque cada contenedor tiene su propio espacio de red: dentro del contenedor de CloudBeaver, `localhost` es el propio CloudBeaver, donde no hay ningún PostgreSQL escuchando.

Al estar los tres contenedores en la red definida por el usuario `red-fis`, Docker ofrece un DNS interno que traduce el nombre del contenedor a su IP (`pg-fis` → 172.25.0.2). Así la conexión no depende de direcciones que pueden cambiar al recrear los contenedores.

Además se usa el puerto **interno** (5432), no el publicado: `-p 5432:5432` solo crea un puente entre el equipo anfitrión y el contenedor, y no interviene en la comunicación dentro de la red.

Si CloudBeaver no estuviera en `red-fis`, quedaría en la red `bridge` por defecto, que no resuelve nombres, y la conexión fallaría con `UnknownHostException: pg-fis`. Habría dos salidas:

1. Conectarlo a la red con `docker network connect red-fis cloudbeaver-fis`.
2. Llegar a la base a través del anfitrión usando el host `host.docker.internal` y el puerto **publicado** (5432 o 3306), lo que funciona en Docker Desktop para Windows y macOS.

Anexo · Comandos usados

Ejecutados en CMD de Windows, en este orden:

```
docker --version
docker compose version
docker info
docker run --rm hello-world

docker network create red-fis

docker pull postgres:18
docker run -d --name pg-fis --network red-fis -e POSTGRES_USER=admin_fis -e
POSTGRES_PASSWORD="Fis2026*Pg" -e POSTGRES_DB=fis_db -p 5432:5432 -v
pg_fis_data:/var/lib/postgresql postgres:18
docker ps --filter name=pg-fis
docker logs pg-fis
docker exec -it pg-fis psql -U admin_fis -d fis_db

docker pull mysql:8.4
docker run -d --name mysql-fis --network red-fis -e
MYSQL_ROOT_PASSWORD="Fis2026*My" -e MYSQL_DATABASE=fis_db -e
MYSQL_USER=app_fis -e MYSQL_PASSWORD="App2026*My" -p 3306:3306 -v
mysql_fis_data:/var/lib/mysql mysql:8.4
docker logs -f mysql-fis
docker exec -it mysql-fis mysql -u root -p

docker pull dbeaver/cloudbeaver:latest
docker run -d --name cloudbeaver-fis --network red-fis -p 8978:8978 -v
cb_fis_workspace:/opt/cloudbeaver/workspace dbeaver/cloudbeaver:latest
docker logs -f cloudbeaver-fis

docker ps
docker network inspect red-fis
```

SQL en PostgreSQL (psql):

```
\du
SELECT rolname, rolsuper, rolcreatedb FROM pg_roles WHERE rolname =
'admin_fis';
CREATE TABLE curso (id SERIAL PRIMARY KEY, nombre VARCHAR(80));
INSERT INTO curso (nombre) VALUES ('Fundamentos de Ingeniería de Software');
SELECT * FROM curso;
```

SQL en MySQL (cliente mysql y CloudBeaver):

```
SELECT user, host FROM mysql.user;
SHOW GRANTS FOR 'root'@'%'\G
SHOW GRANTS FOR 'app_fis'@'%'\G
USE fis_db;
CREATE TABLE curso (id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(80));
INSERT INTO curso (nombre) VALUES ('Arquitectura de Software');
SELECT * FROM curso;
SHOW DATABASES;
```

Conexiones en CloudBeaver: `pg-fis:5432` (admin_fis), `mysql-fis:3306` (root y app_fis) con la propiedad del controlador `allowPublicKeyRetrieval=true`.